**COMP90049 Assignment 2 Research Report on Optical Recognition of Handwritten Digits**

Guangheng (Jack) Tong 1129548

## 1. Introduction

Handwritten digit recognition is a classic problem in machine learning. The task involves classifying images of handwritten digits (0-9) into their corresponding numerical categories. This problem is significant due to its wide range of real-world applications, such as automating postal services for sorting mails, processing bank checks, and digitizing historical documents.

In this project, I utilize the "Optical Recognition of Handwritten Digits" dataset from UCI Machine Learning Repository (https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits). The dataset consists of 5,620 instances, derived from normalized bitmaps of handwritten digits extract from preprinted forms. Originally, these are 32×32 bitmaps, but they are pre-processed by dividing them into 4×4 blocks and counting the number of pixels in each block, resulting in an 8×8 matrix with integer values ranging from 0 to 16. Thus, each instance is described by 64 features, corresponding to these block pixel counts. The dataset is split into a training set (3823 instances) and a test set (1797 instances), contributed by 30 and 13 people, respectively, ensuring a balanced distribution across the 10-digit classes (554 to 572 instances per digit).

This problem captivates me for several reasons. First, it serves as an accessible entry point into the fields of computer vision and pattern recognition, areas I am eager to explore further. Second, the dataset's preprocessing method - reducing the dimensionality from 1024 (=32×32) to 64 (=8×8) – makes it manageable while preserving essential information. At last, I am intrigued by how different machine learning algorithms – ranging from high-bias/low-variance models to low-bias/high-variance models – perform on the same task, especially given the dataset's structured and balanced nature.

## 2. Literature Review

A closely related work is C. Kaynak's (1995) thesis, "Methods of Combining Multiple Classifiers and Their Applications to Handwritten Digit Recognition". This thesis directly utilizes the "Optical Recognition of Handwritten Digits" dataset to explore combining multiple classifiers to enhance recognition accuracy. Kaynak investigates techniques such as voting and stacking, demonstrating that ensemble approaches can outperform individual classifiers by leveraging diverse decision-making strategies.

Additionally, the preprocessing techniques that underpin the dataset's creation are detailed in Garris et al. (1994), "NIST Form-Based Handprint Recognition System" (NISTIR 5469). This report describes the normalization and feature preprocessing technique used to convert 32×32 bitmaps into 8×8 matrices, emphasizing methods that ensure robustness to small distortions. This preprocessing technique is critical to the dataset's utility, as it reduces dimensionality while preserving essential information.

## 3. Methods

**Preprocessing**: The raw dataset, consisting of 64 (=8×8) pixel intensity features (ranging from 0 to 16) per digit, underwent normalization to ensure compatibility with machine learning models. Each of the 64 intensity features was scaled to a [0,1] range using MinMaxScaler from scikit-learn. This is critical for Support Vector Machine (SVM), as their optimization relies on Euclidean distances, which are distorted by unscaled features. For Naïve Bayes, normalization mitigates numerical instability in probability calculations. However, Decision Trees are invariant to monotonic transformations like scaling.

**Feature Construction**: The 64-dimensional vectors are reshaped into 8×8 matrices in order to compute additional spatial features. In the end, 23 additional features were added to capture digit-specific patterns, improving model discriminability:

- Raw Sums (8 features): Sum of pixel intensities per row.
- Column Sums (8 features): Sum of pixel intensities per column.
- Mean Row Indices (1 feature): Weighted average of row indices by intensity. (Vertical "centre of mass")
- Mean Column Indices (1 feature): Weighted average of column indices by intensity. (Horizontal "centre of mass")
- Variance of Row Indices (1 feature): Spread of intensity across rows.
- Variance of Column Indices (1 feature): Spread of intensity across columns.
- High-Intensity Pixel Count (1 feature): Number of pixels > 0.5 (post-normalization).
- Left-Right Symmetry Score (1 feature): Sum of absolute differences between left and right halves.
- Top-Bottom Symmetry Score (1 feature): Sum of absolute differences between top and bottom halves.

**Algorithms**: Three algorithms were implemented using scikit-learn, selected for their complementary strengths.

- Naïve Bayes (High-bias/Low-variance): Assumes feature independence, modelling each feature with a Gaussian distribution. Its simplicity and speed make it ideal as a baseline, though it struggles with correlated features like adjacent pixels. It can handle structured data efficiently with low computational cost.
- SVM (Medium-bias/Medium-variance): Maps data into a higher-dimensional space via the radial basis function (RBF) kernel, excelling at capturing non-linear digit boundaries. It is robust to high-dimensional data like this 87 (=64+23) features dataset (after feature construction).
- Decision Trees (Low-bias/High-variance): Builds a tree by recursively splitting on feature thresholds, capturing complex interactions. It is interpretable and can reveal key features.

**Cross-Validation**: A nested cross-validation framework ensured robust model evaluation.

- Outer Cross-Validation: 5-fold cross-validation divided the dataset (all 5620 instances) into 5 equal parts.
- Inner Cross-Validation: 3-fold cross-validation within each training set tuned hyperparameters.
- Process: For each outer fold, the inner cross-validation tested hyperparameter combinations, selecting the best via accuracy. The outer loop then evaluated this model on the held-out test fold. This nested structure avoids information leakage, as test data remains unseen during tuning.
- Metrics (Accuracy, Precision, Recall, F1-score, Confusion Matrix) are computed per fold, averaged across outer folds for final scores.

**Hyperparameter Tuning**: Grid search within the inner cross-validation optimized hyperparameters.

- Naïve Byes: Tunes var_smoothing = [0.0001, 0.001, 0.01].
    - Higher var_smoothing values add more variance to low-probability regions, improving stability for rare feature values (e.g., all-zero rows).
    - Smaller var_smoothing values keeps the model close to the original Gaussian Naïve Bayes assumption. It works when zero-variance cases are rare.
- SVM: Adjusted C = [5, 7, 10] (regularization strength) and gamma = [0.001, 0.01, 0.1] (RBF kernel parameter).
    - C balances margin size and training error: Small C means wider margin, more tolerance for errors; Large C means narrower margin, less tolerance for errors.
    - Gamma Shapes the decision boundary's flexibility: Small gamma means broad influence, smoother boundary; Large gamma means local influence, more detailed boundary.
- Decision Trees: Optimized max_depth = [10, 15, 20], min_samples_split = [2, 3, 4], min_samples_leaf = [1, 2, 3].
    - max_depth: The maximum depth of decision tree.
    - min_samples_split: The minimum samples to allow a split.
    - Min-samples_leaf: The minimum samples per leaf to prevent overfitting.

The best configurations were selected by maximizing inner cross-validation accuracy, enhancing model fit.

## 4. Results

**Performance metrics**: Table 1 summarizes the performance metrics for each method, including accuracy, precision, recall and F1-score, with standard deviation reflecting variability across cross-validation folds.

Table 1: Performance Metrics for Proposed Methods

| Method | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| **Naive Bayes** | 0.9345 ± 0.0096 | 0.9367 ± 0.0096 | 0.9346 ± 0.0092 | 0.9347 ± 0.0094 |
| **SVM** | 0.9888 ± 0.0022 | 0.9890 ± 0.0019 | 0.9888 ± 0.0023 | 0.9888 ± 0.0021 |
| **Decision Trees** | 0.8993 ± 0.0085 | 0.8996 ± 0.0091 | 0.8991 ± 0.0090 | 0.8987 ± 0.0093 |

**Confusion Matrices**: The below graphs show the confusion matrices for the proposed three methods.

```
[[545.   1.   0.   0.   5.   0.   1.   0.   2.   0.]
 [  0. 484.  23.   1.   0.   0.   4.   1.  31.  27.]
 [  0.   5. 530.   4.   0.   0.   0.   1.  10.   7.]
 [  0.   1.   4. 532.   0.   6.   0.   5.  11.  13.]
 [  0.   1.   0.   0. 526.   1.   5.  16.   9.  10.]
 [  0.   0.   0.   8.   0. 503.   0.   4.   4.  39.]
 [  0.   6.   0.   0.   2.   0. 545.   0.   5.   0.]
 [  0.   1.   0.   1.   0.   6.   0. 551.   2.   5.]
 [  0.  19.   0.   1.   1.   6.   0.   1. 521.   5.]
 [  0.   6.   0.  15.   3.   2.   0.  14.   7. 515.]]
```

Figure 1: Confusion Matrix for Naive Bayes

```
[[552.   0.   1.   0.   0.   0.   1.   0.   0.   0.]
 [  0. 568.   0.   0.   2.   0.   0.   1.   0.   0.]
 [  0.   0. 554.   0.   0.   0.   0.   0.   2.   1.]
 [  0.   0.   0. 563.   1.   2.   0.   1.   2.   3.]
 [  0.   1.   0.   0. 563.   0.   2.   0.   0.   2.]
 [  0.   0.   0.   1.   2. 551.   0.   0.   0.   4.]
 [  0.   2.   0.   0.   1.   0. 555.   0.   0.   0.]
 [  0.   0.   0.   1.   2.   0.   0. 561.   0.   2.]
 [  0.   9.   0.   1.   3.   0.   0.   0. 540.   1.]
 [  0.   1.   0.   3.   2.   1.   1.   1.   3. 550.]]
```

Figure 2: Confusion Matrix for Support Vector Machines

```
[[543.   0.   2.   2.   2.   0.   1.   1.   3.   0.]
 [  1. 513.   9.   6.   5.   4.   4.   4.  22.   3.]
 [  1.   3. 511.  12.   0.   3.   4.   2.  16.   5.]
 [  2.   6.  17. 502.   0.  13.   2.   4.   9.  17.]
 [  6.   9.   3.   0. 530.   4.   4.   3.   1.   8.]
 [  2.   5.   2.   7.   6. 510.   1.   5.   6.  14.]
 [  4.  10.   3.   1.   8.   3. 519.   0.  10.   0.]
 [  1.   5.   3.  15.   5.   5.   0. 521.   3.   8.]
 [  4.  22.  17.  20.   7.  11.   6.   7. 452.   8.]
 [  2.  18.   6.  26.  14.  14.   3.  18.   8. 453.]]
```

Figure 3: Confusion Matrix for Decision Trees

**Hyperparameters**: The best hyperparameter for proposed three methods.

- Naive Bayes: var_smoothing = 0.001
- SVM: C = 5, Gamma = 0.01
- Decision Trees: max_depth = 15, min_samples_leaf = 1, min_samples_split = 3

**Learning Curves**: The below graphs show the learning curves for proposed three methods.

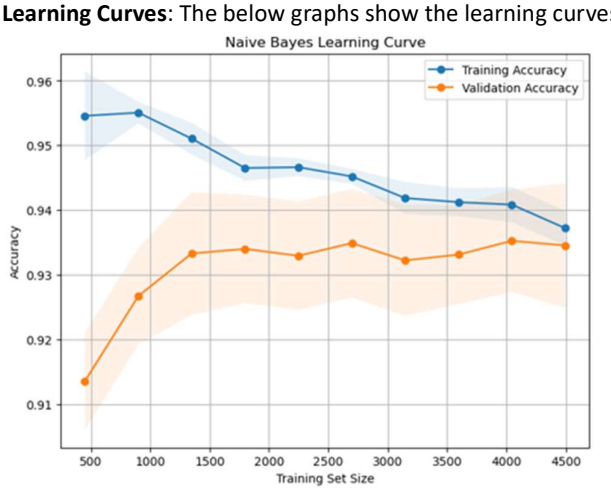


Figure 4: Naive Bayes Learning Curve

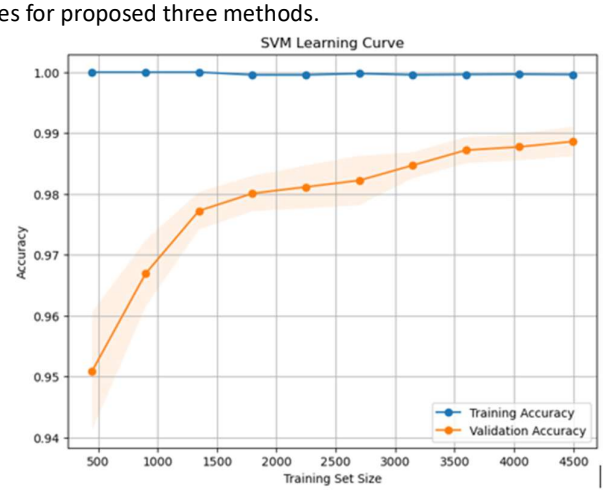


Figure 5: Support Vector Machine Learning Curve

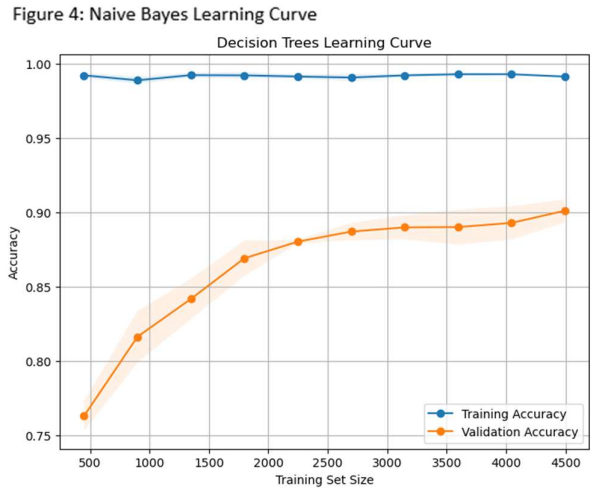


Figure 6: Decision Trees Learning Curve

Naïve Bayes: Achieved an accuracy of 0.9345 ± 0.0096, with balanced precision, recall, and F1-score around 0.935. The confusion matrix shows high diagonal values (correct classifications) but notable misclassifications, such as Class 1 confused with Classes 2 and 9. Training accuracy decreases from 0.954 to 0.937 as the training set size grows from about 450 to 4500, while validation accuracy rises from 0.913 to 0.934. The curve stabilizes around 0.935, suggesting a performance ceiling.

SVM: Outperformed others with an accuracy of 0.9888 ± 0.0022 and near-perfect precision, recall, and F1-score around 0.9889. The confusion matrix shows minimal errors. Training accuracy remains at 1.0 across all set sizes, while validation accuracy improves from 0.95 to 0.989, with a narrowing gap, indicating excellent generalization as dataset size increases.

Decision Trees: Recorded an accuracy of 0.8993 ± 0.0085, with precision, recall, and F1-score around 0.899. The confusion matrix shows good performance but some notable misclassifications. Training accuracy stays near 1.0, while validation accuracy rises from 0.76 to 0.90 as the training set grows from about 450 to 4500, suggesting mild overfitting that diminishes with larger dataset size. The curve seems plateaus at 0.90.

## 5. Discussion

**Analysis and Insights:**

- Naïve Bayes: Misclassifications may arise from correlated features violating the independence assumption, limiting its ability to capture complex patterns.
- SVM: The RBF kernel excels at distinguishing non-linearly separable digits and optimal parameters (C=5, Gamma=0.01) balance fit and generalization.
- Decision Trees: Overfitting (training accuracy ≈ 1.0 vs. validation accuracy ≈ 0.9) suggests it struggles to generalize as well as SVM, especially in high-dimensional spaces.

**Advantages and Disadvantages:**

Naïve Byes:

- Advantages: Computationally efficient, interpretable, and stable. It's ideal as a baseline.
- Disadvantages: Limited by feature independence assumptions, reducing accuracy for correlated features.

SVM:

- Advantages: Handles high-dimensional, non-linear data effectively, offering top-tier accuracy and generalization.
- Disadvantages: Computationally intensive and sensitive to parameter tuning, requiring careful optimization.

Decision Trees:

- Advantages: Interpretable, lightweight, and captures non-linear interactions.
- Disadvantages: Prone to overfitting and less effective in high-dimensional settings compared to SVM.

**Alternative Approaches Considered:** K-Nearest Neighbours was explored for its simplicity but rejected due to poor scalability and high prediction-time costs in high-dimensional spaces.

## 6. Conclusion

This project evaluated three machine learning algorithms – Naïve Bayes, SVM, and Decision Trees – for classifying handwritten digits from 64 intensity features (8×8-pixel images). SVM achieved the highest actuary (0.9888 ± 0.0022), excelling with its RBF kernel to capture non-linear patterns. Naïve Bayes delivered a robust baseline accuracy of 0.9345 ± 0.0096, leveraging its simplicity and feature independence assumption. Decision Trees has a reasonable accuracy of 0.8993 ± 0.0085 but struggled with overfitting.

While SVM offered superior accuracy and generalization, simpler models like Naive Bayes provided efficiency and interpretability, revealing useful trade-offs. Ultimately, this work demonstrates that effective classification hinges on thoughtful algorithm selection, feature enhancement, and parameter tuning. These insights can guide future efforts in tackling complex machine learning challenges.

## Reference

Garris, M. D., Blue, J., Candela, G., Dimmick, D., Geist, J., Grother, P. J., Janet, S. A., & Wilson, C. L. (1994). Nist Form-Based Handprint Recognition System. Nistir 5469. *US Dept. of Commerce, Technology Administration, Nat'l Inst. Standards and Technology*.

Kaynak, C. (1995). *Methods of combining multiple classifiers and their application to handwritten digit recognition*.